

Algoritmi e Strutture Dati

31 gennaio 2024

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti) Realizzare una funzione booleana di tipo divide et impera $\text{Ord}(A, p, r)$ che verifica se l'array $A[p, r]$ è ordinato in senso crescente. Scrivere lo pseudocodice e valutare la complessità con il master theorem.

Domanda B (6 punti) Si consideri una tabella hash di dimensione $m = 8$, gestita mediante chaining (liste di trabocco) con funzione di hash $h(k) = k \bmod m$. Si descriva in dettaglio come avviene l'inserimento della sequenza di chiavi: 13, 10, 33, 21, 8, 26

Esercizi

Esercizio 1 (10 punti) Realizzare una funzione $\text{Prod}(A, k)$ che dato un array A di interi ≥ 0 ordinato in senso crescente e un valore intero $k \geq 0$ verifica se esistono due indici i e j tali che $k = A[i] * A[j]$. Valutarne la complessità. Adattare la soluzione al caso in cui i valori nell'array possono essere negativi (assumendo ancora $k \geq 0$).

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori s e f di tempi di inizio e fine, e ordinate per tempo di *inizio* (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- (a) Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- (b) Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 2, 3, 5, 7, 10) \quad f = (3, 9, 10, 7, 11, 12)$$

- (c) Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.

Algoritmi e Strutture Dati

14 febbraio 2024

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale fornire una spiegazione dell'idea che questo segue.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia o brutta copia*.

Domande

Domanda A (7 punti) Si dia la definizione di limite asintotico stretto. Data la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2T(n/5) + T(n/2) + n & \text{se } n > 1 \end{cases}$$

mostrare che $f(n) = n$ è limite asintotico stretto per la soluzione.

Domanda B (6 punti) Indicare, in forma di albero binario, il codice prefisso ottenuto tramite l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f\}$, supponendo che ogni simbolo appaia con le seguenti frequenze:

a	b	c	d	e	f
11	6	13	35	10	25

Spiegare brevemente il processo di costruzione del codice.

Esercizi

Esercizio 1 (10 punti) Un ricco possidente deve lasciare in eredità le sue $2n$ proprietà a n figli. Il valore delle proprietà è contenuto in un array di numeri (reali positivi) $A[1..2n]$.

1. Sviluppare un algoritmo $\text{Split}(A, n)$ che dato in input l'array $A[1..2n]$ dei valori delle proprietà e numero n , verifica se le $2n$ proprietà possano partizionate in n coppie, tutte con lo stesso valore complessivo (di modo da assegnare una coppia di proprietà a ciascun figlio). Si assuma che $n > 0$.
2. Si valuti la complessità dell'algoritmo proposto.
3. Si dimostri la correttezza dell'algoritmo proposto.

Esercizio 2 (9 punti) Data una stringa $X = \langle x_1, x_2, \dots, x_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

- (a) Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{RECL}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.
- (b) Determinarne la complessità al caso migliore $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Algoritmi e Strutture Dati

18 giugno 2024

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale fornire una spiegazione dell'idea che questo segue.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti) Dare una soluzione asintotica per la ricorrenza $T(n) = 4T(n/2) + n^3 + 1$.

Domanda B (6 punti) Dare la definizione di max-heap. Dato l'array A con elementi 7, 1, 17, 0, 5, 4, 22, si specifichi il max-heap ottenuto applicando ad A la procedura `BuildMaxHeap`. Si descriva sinteticamente come si procede per arrivare al risultato (ad esempio illustrando come opera `BuildMaxHeap` e riportando il contenuto dello heap nei passi intermedi).

Esercizi

Esercizio 1 (9 punti) Realizzare una procedura `triplet(A)` che dato un array $A[1,n]$ di interi verifica se esistono tre indici, non necessariamente distinti, i , j e k tali che $A[i] + A[j] = A[k]$. Fornire lo pseudocodice, motivare la correttezza della soluzione e valutarne la complessità.

Esercizio 2 (11 punti) Una *longest common substring* di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una *longest common substring*. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

- (a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?
- (b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m + n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
- (c) Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. (Suggerimento: considerare la lunghezza della *longest common substring* dei prefissi $X_i = \langle x_1, \dots, x_i \rangle$ e $Y_j = \langle y_1, \dots, y_j \rangle$ che termina con x_i e y_j , rispettivamente.)

Algoritmi e Strutture Dati

2 luglio 2024

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale fornire una spiegazione dell'idea che questo segue.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti)

1. Dare la definizione della notazione Ω , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = \Omega(g(n))$.
2. Ordinare le seguenti funzioni per ordine di grandezza decrescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = \Omega(f_2), f_2 = \Omega(f_3), \dots, f_7 = \Omega(f_8)$.

$$n^{2/3} \quad 10 \quad \frac{n}{\sqrt{n}} \quad 1.1^n \quad \frac{n}{2^n} \quad n^2 \quad \sqrt{\log n} \quad \log n$$

Domanda B (6 punti) Scrivere la ricorrenza sulle lunghezze $\ell(i, j)$ per il problema della longest common subsequence (LCS).

Esercizi

Esercizio 1 (10 punti) Realizzare una funzione $\text{union}(A1, A2, n)$ che dati due array di interi $A1$ e $A2$, organizzati a max-heap, con capacità n , restituisce un nuovo array A , ancora organizzato a max-heap con capacità $2n$, che contiene l'unione insiemistica dei valori contenuti in $A1$ e $A2$. Si assuma che $A1$ e $A2$ non contengano duplicati e si faccia in modo anche l'array ottenuto come unione non contenga duplicati. Ad es. se $A1$ contiene i valori 3, 1, 2 e $A2$ contiene i valori 5, 2 allora l'unione A conterrà i valori 5, 3, 1, 2, possibilmente non in questo ordine, ovvero l'elemento 2 non è duplicato. Valutare la complessità della funzione definita.

Qualora il risultato A potesse contenere duplicati ci sarebbero soluzioni più efficienti?

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori s e f di tempi di inizio e fine, e ordinate per tempo di *inizio* (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- (a) Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- (b) Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 2, 3, 6, 7, 9) \quad f = (4, 5, 7, 8, 12, 11)$$

- (c) Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.

Algoritmi e Strutture Dati

19 settembre 2024

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale fornire una spiegazione dell'idea che questo segue.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Definire formalmente la classe $O(f(n))$. Dimostrare che la seguente ricorrenza ha soluzione $T(n) = O(n)$

$$T(n) = \frac{1}{2}T(n-1) + 3n + 1$$

Domanda B (7 punti) Dare la definizione di max-heap. Data la sequenza di elementi 17, 19, 22, 15, 30, 24, 23, si specifichi il max-heap ottenuto inserendo, a partire da uno heap vuoto, uno alla volta questi elementi nell'ordine indicato e infine rimuovendo 15. Si descriva sinteticamente come si procede per arrivare al risultato.

Esercizi

Esercizio 1 (9 punti) In questo esercizio si considerano alberi binari di ricerca (BST) completi, memorizzati in array con la stessa convenzione che si usa normalmente per la memorizzazione di uno heap in un array.

Realizzare una funzione $\text{max}(T, n)$ che determina il massimo di un array T di dimensione n che memorizza un BST completo.

Scrivere quindi una funzione $\text{merge}(T1, T2, k)$ che dati due array $T1$ e $T2$ di dimensione n , che memorizzano BST completi e una chiave k maggiore di tutte le chiavi di $T1$ e minore di tutte quelle di $T2$, restituisce un array (di dimensione $2n+1$) che memorizza un BST che contiene tutte le chiavi di $T1$ e $T2$, e la chiave k .

In entrambi i casi motivare la correttezza delle funzioni e valutarne la complessità.

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $a, b \in \mathbb{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j-1) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $m = \max\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

- (a) Scrivere un algoritmo iterativo bottom-up per il calcolo di m .
- (b) Valutare la complessità *esatta* dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Algoritmi e Strutture Dati

24 gennaio 2025

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Dare la definizione di max-heap. Dato un insieme S di elementi, memorizzato in parte in un min-heap A e in parte in un max-heap B , entrambi non vuoti, dare un algoritmo $\min(A, B)$ per trovare il minimo di S nelle due situazioni seguenti:

- (a) ogni elemento di A è minore o uguale a ogni elemento di B ;
- (b) ogni elemento di B è minore o uguale a ogni elemento di A .

In entrambi i casi scrivere lo pseudo-codice e valutare la complessità.

Domanda B (7 punti) Si consideri il problema di selezione di attività compatibili:

- (a) Definire il problema.
- (b) Descrivere brevemente l'algoritmo ottimo GREEDY-SEL visto in classe.
- (c) Fornire un esempio di algoritmo greedy *non* ottimo, motivandone la non ottimalità.

Esercizi

Esercizio 1 (10 punti) Sia dato un array $V[1..n]$ i cui valori rappresentano la variazione giornaliera del valore di un titolo azionario. È noto che il titolo è stato prima perdita, con valori sempre negativi, poi ha iniziato a oscillare in giorni *consecutivi* tra valori positivi e negativi, e infine si è stabilizzato su valori positivi (dunque nella sequenza non ci possono essere due giorni positivi seguiti da un negativo). Realizzare un algoritmo *divide et impera* $\text{Split}(V)$ che individua il giorno in cui il titolo ha iniziato a essere stabile su valori positivi, ovvero il minimo indice $i \in [1, n]$ tale che per ogni $j \geq i$ vale $V[j] > 0$. Se il titolo non si stabilizza su valori positivi, ritornare 0. Ad es., se l'array $V = [-1, -2, 2, -1, 6, 3]$ l'indice da tornare sarà 5, mentre invece per $V = [-1, -2, 2, -1, 6, -3]$ si ritornerà 0. Fornire lo pseudocodice di $\text{Split}(V)$, motivarne la correttezza e individuarne la complessità. Si assuma che non ci siano valori nulli.

Esercizio 2 (8 punti) Date due stringhe $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per ogni coppia di valori i, j con $0 \leq i \leq m$ e $0 \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

Si vuole calcolare la quantità $q = \max\{\ell(i, j) : 0 \leq i \leq m, 0 \leq j \leq n\}$.

- (a) Scrivere un algoritmo bottom-up per il calcolo di q .
- (b) Determinare la complessità *esatta* dell'algoritmo, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Algoritmi e Strutture Dati

7 febbraio 2025

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (5 punti) Si determini la soluzione asintotica della seguente equazione di ricorrenza:

$$T(n) = 3T(n/3) + n^2 + 1$$

Domanda B (7 punti) Si consideri una tabella hash di dimensione $m = 7$, e indirizzamento aperto con doppio hash basato sulle funzioni $h_1(k) = k \bmod m$ e $h_2(k) = 1 + k \bmod (m - 2)$. Si descriva sinteticamente come avviene l'inserimento degli elementi e si specifichi il risultato dell'inserzione della sequenza di chiavi: 10, 20, 34, 35, 48.

Sarebbe appropriato lavorare con una tabella di dimensione $m = 8$ e le stesse funzioni hash?

Esercizi

Esercizio 1 (10 punti) Siano dati due array $A[1..2n]$ e $B[1..n]$ organizzati a max-heap, entrambi contenenti n elementi (**heapsize=n**). Realizzare una procedura **SortJoin(A,B,n)** che dati in input array A e B con le proprietà sopra descritte, ritorna in A un array ordinato contenente tutti i $2n$ elementi originariamente presenti in A e B . L'array B può essere modificato durante l'esecuzione della procedura, se necessario, ma l'algoritmo dovrà operare in *spazio costante*. Dare lo pseudocodice della procedura, motivarne la correttezza e valutarne la complessità. Se si utilizzano operazioni sui max-heap andranno definite esplicitamente.

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine, e ordinate per tempo di *inizio* (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- (a) Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- (b) Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (1, 2, 3, 5, 7, 10) \quad \mathbf{f} = (3, 9, 10, 7, 11, 12)$$

- (c) Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.

Algoritmi e Strutture Dati

18 giugno 2025

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Dare la definizione formale delle classi $O(f(n))$ e $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che se $f(n) = O(n)$ e $g(n) = n^2 - f(n)$ allora $g(n) = \Omega(n^2)$.

Domanda B (6 punti) Indicare, in forma di albero binario, il codice prefisso ottenuto tramite l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f\}$, supponendo che ogni simbolo appaia con le seguenti frequenze:

a	b	c	d	e	f
12	7	14	30	10	27

Spiegare brevemente il processo di costruzione del codice.

Esercizi

Esercizio 1 (9 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , in *tempo costante*, il numero delle foglie nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `leaves(x)` che restituisce il numero delle foglie nel sottoalbero radicato in x e per la procedura di inserimento di un nodo `insert(T, z)`. Per entrambe valutare la complessità.

Esercizio 2 (10 punti) Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza ℓ_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione. Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{j_i}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$. L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_{j=1}^n C_j(\sigma)$.

- (a) Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
- (b) Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Algoritmi e Strutture Dati

10 settembre 2025

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (7 punti) Dare la definizione formale della classe $\Omega(f(n))$ per una funzione $f(n)$. Mostrare che date due funzioni $f(n)$, $g(n)$ (che si possono assumere sempre positive), se $f(n) = \Omega(n^2)$ allora $f(n) + g(n) = \Omega(n^2 + g(n))$. Vale anche $f(n) - g(n) = \Omega(n^2 - g(n))$? Dimostrarlo o fornire un controesempio.

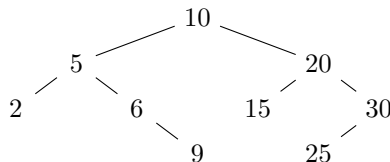
Domanda B (6 punti) Indicare, in forma di albero binario, il codice prefisso ottenuto tramite l'algoritmo di Huffman per l'alfabeto $\{a, b, c, d, e, f\}$, supponendo che ogni simbolo appaia con le seguenti frequenze:

a	b	c	d	e	f
12	7	14	30	10	27

Spiegare brevemente il processo di costruzione del codice.

Esercizi

Esercizio 1 (10 punti) Realizzare una funzione `mdist(T,v)` che dato un albero binario di ricerca T e una chiave v restituisce un nodo x di T la cui chiave abbia distanza minima da v , ovvero tale che il valore $|x.k - v|$ sia minimo. Fornire lo pseudocodice, valutarne la complessità e giustificarne la correttezza. Ad esempio se T è l'albero sotto indicato, allora `mdist(T,8)` ritorna il nodo con chiave 9 mentre `mdist(T,28)` ritorna il nodo con chiave 30.



Esercizio 2 (9 punti) Sia $n > 0$ un intero. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 1 & \text{se } i = j, \\ 2 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) \cdot M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

- (a) Scrivere una coppia di algoritmi `INIT-M(n)` e `REC-M(i, j)` per il calcolo memoizzato di $M(1, n)$.
- (b) Calcolare il numero *esatto* $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Algoritmi e Strutture Dati

20 gennaio 2026

Note:

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano il testo e tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti)

1. Dare la definizione della notazione O , cioè, date due funzioni $f(n)$ e $g(n)$, definire il significato della notazione $f(n) = O(g(n))$.
2. Ordinare le seguenti funzioni per ordine di grandezza crescente, cioè scrivere le funzioni secondo un ordine $f_1, f_2, \dots, f_8$ tale che risulti $f_1 = O(f_2), f_2 = O(f_3), \dots, f_7 = O(f_8)$.

$$n^{3/2} \quad 2^n \quad n \log^2 n \quad \log n \quad n^{0.9} \quad (\log n)^3 \quad 1.05^n \quad \frac{n^2}{\log n}$$

Domanda B (6 punti) Dare la definizione di max-heap. Dato l'array A con elementi 7, 1, 17, 0, 5, 4, 22, si specifichi il max-heap ottenuto applicando ad A la procedura BuildMaxHeap. Si descriva sinteticamente come si procede per arrivare al risultato (ad esempio illustrando come opera BuildMaxHeap e riportando il contenuto dello heap nei passi intermedi).

Esercizi

Esercizio 1 (10 punti) Realizzare un arricchimento degli alberi binari di ricerca che consenta di ottenere per ogni nodo x dell'albero, in *tempo costante*, la massima differenza tra chiavi presenti nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `diff(x)` che restituisce la massima differenza tra chiavi presenti nel sottoalbero radicato in x e per la procedura di inserimento di un nodo `insert(T, z)`. Per entrambe valutare la complessità.

Esercizio 2 (9 punti) Avete n file $f_1, f_2, \dots, f_n$ di dimensioni $d_1, d_2, \dots, d_n$ MB da caricare sul vostro servizio di archiviazione cloud. Purtroppo la somma delle dimensioni di questi file è superiore alla capacità di c MB disponibile su cloud, e non volete pagare per ottenere più spazio. Vi dovete quindi accontentare di caricare il maggior numero possibile di file, senza eccedere la capacità c .

- (a) Progettare un algoritmo greedy per questo problema, e valutarne la complessità.
- (b) Enunciare e dimostrare la proprietà di scelta greedy su cui l'algoritmo si basa.

Algoritmi e Strutture Dati

4 luglio 2025

Note

1. La leggibilità è un prerequisito: parti difficili da leggere potranno essere ignorate.
2. Quando si presenta un algoritmo è fondamentale spiegare l'idea e motivarne la correttezza.
3. L'efficienza e l'aderenza alla traccia sono criteri di valutazione delle soluzioni proposte.
4. Si consegnano tutti i fogli, con nome, cognome, matricola e l'indicazione *bella copia* o *brutta copia*.

Domande

Domanda A (6 punti) Si dimostri che la ricorrenza che segue ha soluzione $T(n) = \Theta(n)$

$$T(n) = \frac{2}{3}T(n-1) + 2n$$

Domanda B (7 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi: 10, 5, 3, 15, 7, 12. Si supponga che dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Esercizi

Esercizio 1 (10 punti) Dato un array $A[1, n]$ di interi un indice i si dice *stabile* se $A[i] = i$. Realizzare una procedura `stab(A, n)` che dato in input un array $A[1, n]$ di interi *distinti*, ordinato in modo crescente, ritorna un indice stabile, se esiste, e ritorna 0 altrimenti. Dimostrarne la correttezza e valutarne la complessità.

Esercizio 2 (9 punti) Data una stringa $X = \langle x_1, x_2, \dots, x_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

- (a) Scrivere una coppia di algoritmi `INIT L(X)` e `REC L(X, i, j)` per il calcolo memoizzato di $\ell(1, n)$.
- (b) Determinarne la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.